Analisis Algoritme Pertemuan 2

Slide disarikan dari berbagai sumber

Tujuan Pertemuan 2

- Mahasiswa dapat menjelaskan
  - kriteria kebaikan suatu algoritme
  - laju pertumbuhan fungsi suatu algoritme
  - notasi asimtotik
  - aproksimasi kompleksitas algoritme menggunakan notasi asimptotik
  - menggolongkan fungsi berdasar laju pertumbuhannya

Kriteria Kebaikan Algoritme

- **Correctness**: apakah suatu algoritme selalu menghasilkan output yang benar untuk setiap kemungkinan input?
- **Amount of Work Done (Time Complexity)**: jumlah pekerjaan yang dilakukan untuk menghasilkan output jika diberikan input dengan ukuran tertentu.
- **Amount of Space Used (Space Complexity)**: jumlah memori yang dilakukan untuk menghasilkan output jika diberikan input dengan ukuran tertentu.
- **Simplicity / Clarity**: seberapa mudah suatu algoritme dipahami, dibaca, dan diimplementasikan dalam bentuk program.
- **Optimality**: algoritme tersebut adalah yang terbaik untuk menyelesaikan masalah dengan batasan masalah yang diberikan.

Correctness

Algoritma menyelesaikan masalah dengan tepat, sesuai dengan definisi masukan / keluaran yang diberikan

Deskripsi Algoritme

Masalah, persoalan yang ingin diselesaikan

Masukan, data yang menjadi permasalahan

Keluaran, bentuk akhir dari data setelah algoritma diimplementasikan

Correctness

Contoh mendefinisikan sebuah Algoritme :

- Masalah
  - Pengurutan sekumpulan nilai yang bernilai acak.
- Masukan
  - Serangkaian data berukuran n.
- Keluaran
  - Serangkaian data berukuran n, dengan urutan $a_1 \leq a_2 \leq a_3 \leq \cdots \leq a_{n-1} \leq a_n$, di mana a_x adalah data pada posisi x dalam rangkaian.

Correctness

Pembuktian Formal: Induksi matematika

1. Melakukan pembuktian *base case :*
   - algoritma bernilai benar jika diaplikasikan pada bilangan pertama yang sah sesuai dengan spesifikasi fungsi atau algoritma tersebut.

2. Melakukan induksi, yaitu membuktikan fungsi P(k+1) benar jika fungsi P(k) benar

Correctness

- Reminder : pembuktian dengan induksi matematika

Buktikan bahwa pernyataan matematika untuk perhitungan deret aritmatika berikut:

$$1 + 2 + 3 + \ldots + n = \frac{n(n+1)}{2}$$

adalah benar untuk semua bilangan bulat n≥1.

$$1+2+3+\ldots+n=\frac{n(n+1)}{2}$$

Correctness

1. Melakukan pembuktian *base case* :

$$P(1)=1=\frac{1(1+1)}{2}$$

$$1=\frac{1(2)}{2}$$

$$1=\frac{2}{2}$$

$$1=1$$ Benar

2. Melakukan induksi:

$$1+2+3+\ldots+n=\frac{n(n+1)}{2} \Rightarrow P(k)=1+2+3+\ldots+k=\frac{k(k+1)}{2}$$ Benar

$$P(k+1)=1+2+\ldots+k+(k+1)=\frac{(k+1)((k+1)+1)}{2}$$ Benar ???

$$P(k+1) = 1 + 2 + \ldots + k + (k+1) = \frac{(k+1)((k+1)+1)}{2}$$

$$1 + 2 + \ldots + k + (k+1) = (1 + 2 + \ldots + k) + (k+1)$$

$$P(k) = 1 + 2 + 3 + \ldots + k = \frac{k(k+1)}{2}$$

$$= \frac{k(k+1)}{2} + (k+1)$$

$$= \frac{k(k+1) + 2(k+1)}{2}$$

$$= \frac{k^2 + 3k + 2}{2}$$

$$= \frac{(k+1)(k+2)}{2}$$

$$= \frac{(k+1)((k+1)+1)}{2}$$

Correctness

- induksi matematika sangat berguna untuk membuktikan kebenaran fungsi yang melibatkan perhitungan berulang.
- Nyatakan algoritme sebagai sebuah fungsi berulang

```
[1]  s = 0
[2]  i = 0
[3]  while (i<n) do
[4]            s = s + n
[5]            i = i+1
[6]  end while
[7]  return s
```



Ex: n=3

Now i =0, since i < 3, enter loop first time:

$s \leftarrow 0 + 3$

$i \leftarrow 0 + 1$

producing s = 3 and i = 1.

Now i = 1, since i < 3, enter loop second time:

$s \leftarrow 3 + 3$

$i \leftarrow 1 + 1$

producing s = 6 and i = 2.

Correctness

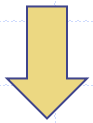
Now i=2, since i < 3, enter loop third time :

s <- 6 + 3

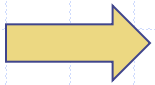
i <- 2 + 1

producing s = 9 and i = 3.

Now i=3, Since i = 3, the while loop is not entered any longer, S = 9 is returned and the algorithm is terminted.

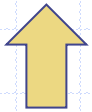


Untuk n=3,
Loop 1 : s=1*3, i=1
Loop 2 : s=2*3, i=2
Loop 3 : s=3*3, i=3



after going through the loop **k** times, **s = k*n** and **i = k** hold

Loop Invariant



Correctness

after going through the loop **k** times, **s = k*n** and **i = k** hold

1. Basis Step: k = 0. When k = 0, that is when the loop is not entered, s = 0 and i = 0. Hence s = k*n and i = k hold.

2. Induction Hypothesis: For an arbitrary value m of k, s = m * n and i = m hold after going through the loop m times.

3. Inductive Step: When the loop is entered (m + 1)-st time, then

   prove s = (m+1) * n and i = m+1

Remember this



s = s + n
i = i+1

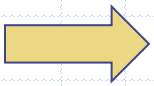
s = m*n and i = m at the beginning of the loop. Inside the loop,

    s <- m*n + n
    i <- m + 1

producing s = (m + 1)*n and i = m + 1, Thus S = k*n and i = k hold for any natural number k.

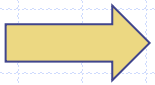
Correctness

```
[1]  p = 1
[2]  i = 1
[3]  while (i<=n) do
[4]            p = p*i
[5]            i = i+1
[6]  end while
[7]  return p
```



For example n=3 :

- p← 1, i←2
- p←2, i←3
- p←6, i←4



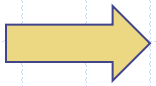
show that after going through the loop **k** times;
p = k ! and i = k + 1 hold.

Correctness

```
[1]  p = 1
[2]  i = 1
[3]  while (i<=n) do
[4]          p = p*i
[5]          i = i+1
[6]  end while
[7]  return p
```

show that after going through the loop **k** times; **p = k ! and i = k + 1** hold.

1. Basis Step: k = 1.
   - p = 1 * 1 = 1 , i = 1 + 1 = 2.
   - p = k! and i = k + 1 hold.
2. Induction Hypothesis: For k=m, after going through the loop m times.
   - p = m! , i = m + 1 hold
3. Inductive Step: For k=m+1, proof p = (m + 1)! and i = (m + 1) + 1.
   - m-loop : p = m! , i = (m+1)

     remember  p = p*i, i = i+1
   - m+1-loop : p = m! * (m + 1) , i = (m + 1) + 1
   - after going through the loop m+1 times.

→ p = (m + 1)! and i = (m + 1) + 1.

Efficiency

- Dua hal yang sering diukur untuk menilai effisiensi:

  1. Time: berapa lama sebuah algoritme bekerja?

  2. Space: berapa banyak ukuran memory yang dibutuhkan oleh algoritme?

Perbandingan Empiris

- Membandingkan algoritme berdasarkan lamanya *runtime*
- Asumsi: lingkungan komputer persis sama untuk setiap algoritme yang dijalankan (mis: bahasa pemrograman dan komputer).

Run time (nanoseconds)		$1.3\ N^3$	$10\ N^2$	$47\ N \log_2 N$	$48\ N$
Time to solve a problem of size	1000	1.3 seconds	10 msec	0.4 msec	0.048 msec
	10,000	22 minutes	1 second	6 msec	0.48 msec
	100,000	15 days	1.7 minutes	78 msec	4.8 msec
	million	41 years	2.8 hours	0.94 seconds	48 msec
	10 million	41 millennia	1.7 weeks	11 seconds	0.48 seconds
Max size problem solved in one	second	920	10,000	1 million	21 million
	minute	3,600	77,000	49 million	1.3 billion
	hour	14,000	600,000	2.4 trillion	76 trillion
	day	41,000	2.9 million	50 trillion	1,800 trillion
N multiplied by 10, time multiplied by		1,000	100	10+	10

Kompleksitas

- Membandingkan algoritme pada level konsep.
- Tidak berpengaruh pada:
  - Bahasa pemrograman yang digunakan.
  - Perangkat keras dan lingkungan yang digunakan.
- Mengukur banyaknya operasi (f(*n*))yang dilakukan berdasarkan ukuran input (*n*) yang diberikan.
  - Operasi: yang paling dominan dalam suatu algoritme (operasi matematika, perbandingan, jumlah kueri, dll).
  - Input: panjang list, dimensi matriks, dll
- Sangat berguna untuk memprediksi perilaku algoritme ketika ukuran *input* sangat besar → *runtime* bisa jadi terlalu lama untuk dibandingkan dengan perbandingan empiris.

Ukuran Input dan Operasi Dasar

Problem	Ukuran Input	Operasi Dasar

Menghitung F(n)

Example: Simple If-Statement

	Cost	Times
`if (n < 0)`	c1	1
`  absval = -n`	c2	1
`else`		
`  absval = n;`	c3	1

Total Cost = c1 + max(c2,c3)

Example: Simple Loop

	Cost	Times
`i = 1;`	c1	1
`sum = 0;`	c2	1
`while (i <= n) {`	c3	n+1
`  i = i + 1;`	c4	n
`  sum = sum + i;`	c5	n
`}`		

Total Cost = c1 + c2 + (n+1)*c3 + n*c4 + n*c5

Example: Nested Loop

	Cost	Times
`i=1;`	c1	1
`sum = 0;`	c2	1
`while (i <= n) {`	c3	n+1
`  j=1;`	c4	n
`  while (j <= n) {`	c5	n*(n+1)
`  sum = sum + i;`	c6	n*n
`  j = j + 1;`	c7	n*n
`  }`		
`  i = i +1;`	c8	n
`}`		

Total Cost = c1 + c2 + (n+1)*c3 + n*c4 + n*(n+1)*c5+n*n*c6+n*n*c7+n*c8

Jika dianggap semua cost bernilai 1 satuan waktu transaksi, maka berapakah F(n) dari masing-masing potongan program?

Menghitung F(n)

	1 instruksi → 1 satuan waktu (pukul rata)		asumsin waktu eksekusi : Assign, operasi boolean: → ½ satuan waktu Aritmetika: →1 satuan waktu Looping for – next, operasi I/O : → 2 satuan waktu
	Misal n=4	n	
Algoritma Rata-1 : real Begin 1 Jml ⇐ 0 ---------- 2 For i ⇐1 to n ---------- 3 Do jml ⇐ jml + A[i] ---------- End-for 4 Rata_1 ⇐ jml/n ---------- End-Alg1			

Algoritma Rata-2 : real Begin 1 i ⇐ 1 ---------- 2 Jml ⇐ 0 ---------- 3 While i <= n ---------- 4 Do jml <= jml + A[i] ---------- 5 i ⇐ i + 1---------- End-while 6 Rata_2 ⇐ jml/n ---------- End-Alg2			

Sumber : https://lms--paralel-esaunggul-ac-id.webpkgcache.com/doc/-/s/lms-paralel.esaunggul.ac.id/mod/resource/view.php?id=216807

F(n) pada Average Case, Best Case, Worst Case

Mencari berapa banyak bilangan yang bernilai < x pada array A[i]

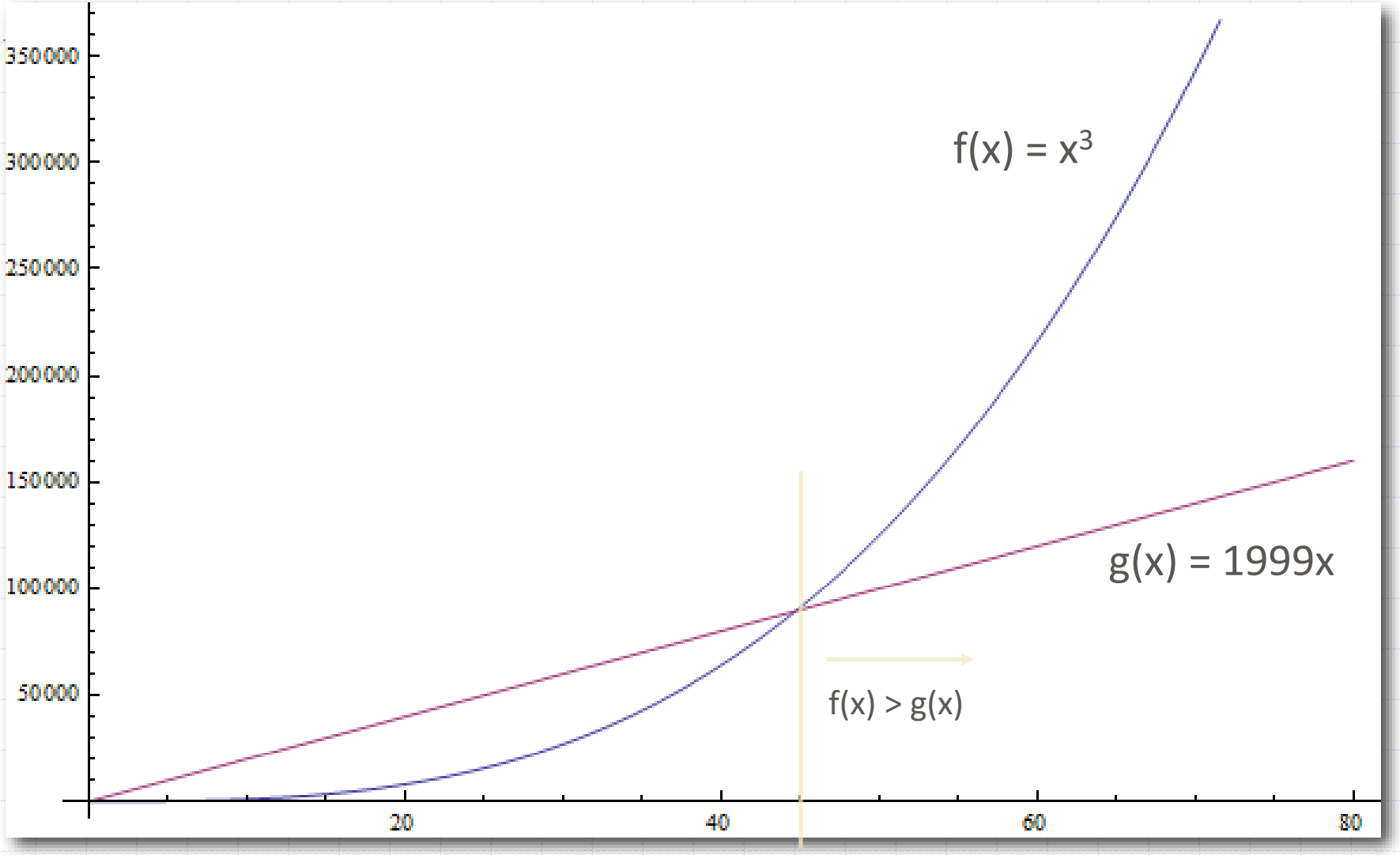
Algoritma Cari Nilai_lk_x Begin 1 i ⇐ 1 ---------- 2 Jml ⇐ 0 ---------- 3 While i <= n ---------- Do 4 IF A [i] < x 5 Then jml <= jml + 1 End-if 6 i ⇐ i + 1---------- End-while 7 Write (jml) End-Alg2	1 x ½ = ½ 1 x ½ = ½ (n+1)x ½ = ½n+½ n x ½ = ½n 50%n x 1½ = ¾ n n x 1½ = 1½n 1 x 2 = 2 f(n) = 3¼ n + 3½	Analisa keadaan Keadaan rata-rata kondisi IF (4) bisa benar, bisa salah maka instruksi baris 5 memiliki peluang → 50% f(n) = 3¼ n +3½ (AVERAGE CASE) Jika tidak ada data yang lebih kecil dari x, sehingga kondisi IF selalu salah maka intsruksi 5 tdk pernah dikerjakan → f(n) = 2½ n + 3½ (BEST CASE) Jika semua data lebih kecil dari x, sehingga kondisi IF selalu benar maka intsruksi 5 selalu dikerjakan (100%) → f(n) = 4n + 3½ (WORSTCASE)

Sumber : https://lms--paralel-esaunggul-ac-id.webpkgcache.com/doc/-/s/lms-paralel.esaunggul.ac.id/mod/resource/view.php?id=216807

Sifat Asimtotik

- f(n) sangat membantu kita memahami seberapa baik suatu algoritme. Namun, pada praktiknya, f(n) dapat disederhanakan.
- Cukup perhatian bagian yang bergantung pada ukuran input.
- Misal $f(n) = 6n + 4$.
  - Cukup perhatikan 6n sehingga $f(n) = 6n$.
- Bagaimana jika $f(n) = 3n^2 + 6n + 12$?
  - Pilih suku (*term*) yang pertumbuhannya paling cepat: $3n^2$
- Konstanta pada suku pun bisa dihilangkan sehingga $f(n) = 5n^3 + 1999n$ dapat dianggap setara dengan $f(n) = n^3$.
  - $f(n) = n^3$ mendeskripsikan sifat asimtotik dari $f(n) = 5n^3 + 1999n$.
- Growth Rates : Laju pertumbuhan fungsi/Laju pertumbuhan asimtotik

Sifat Asimtotik



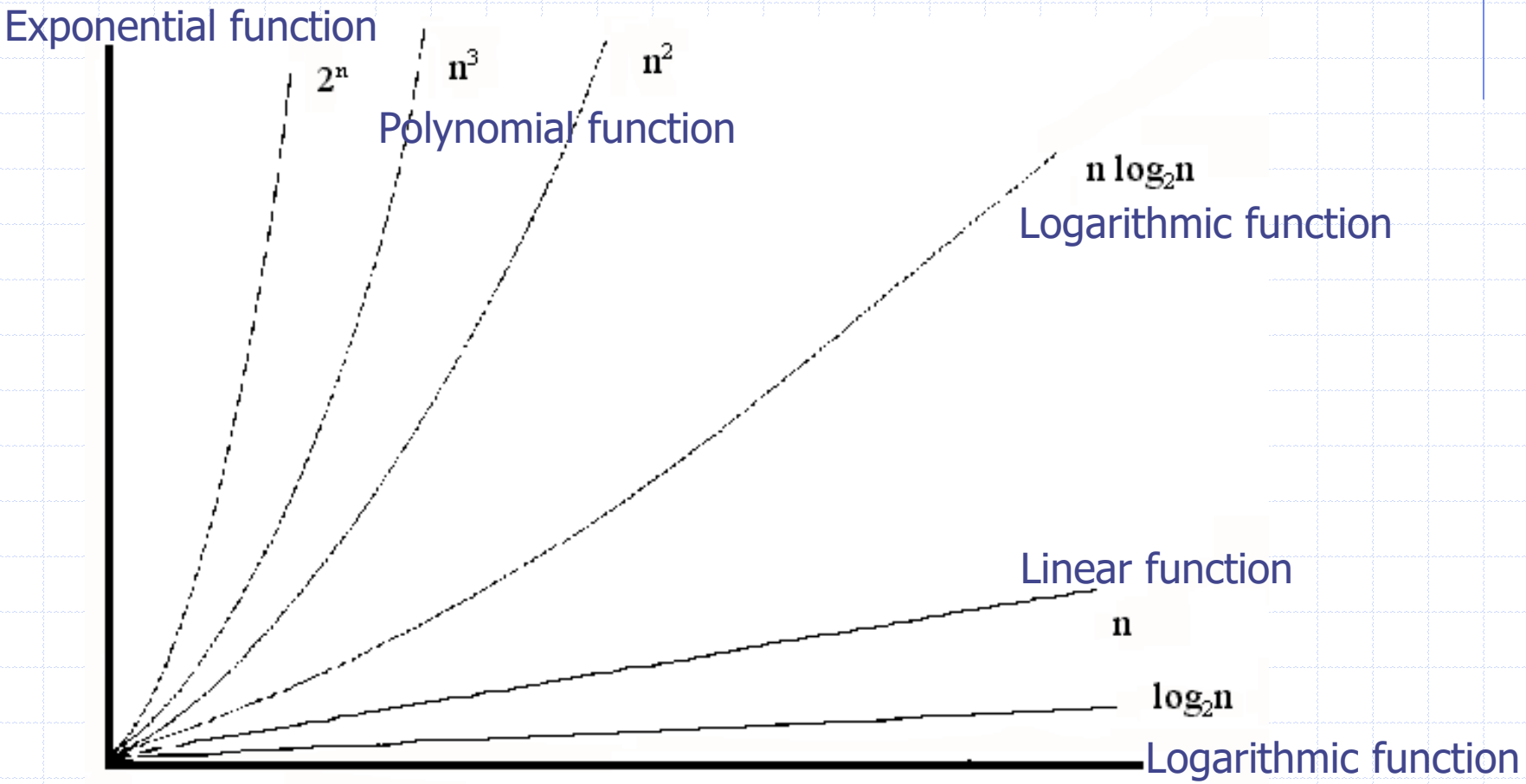
Big O Notation

- If *Algorithm A requires time proportional to f(n),* Algorithm A is said to be **order f(n),** and it is denoted as **O(f(n)).**
- The **function f(n)** is called the algorithm's **growth-rate function**.
- Since the capital O is used in the notation, this notation is called the **Big O notation**.
- If Algorithm A requires time proportional to $\mathbf{n^2}$, it is $\mathbf{O(n^2)}$.
- If Algorithm A requires time proportional to **n**, it is **O(n).**

The Growth Rate of the Six Popular Functions

n	$\log n$	$n\log n$	n^2	n^3	2^n
4	2	8	16	64	16
8	3	24	64	512	256
16	4	64	256	4,096	65,536
32	5	160	1,024	32,768	4,294,967,296
64	6	384	4,094	262,144	$1.84 * 10^{19}$
128	7	896	16,384	2,097,152	$3.40 * 10^{38}$
256	8	2,048	65,536	16,777,216	$1.15 * 10^{77}$
512	9	4,608	262,144	134,217,728	$1.34 * 10^{154}$
1024	10	10,240	1,048,576	1,073,741,824	$1.79 * 10^{308}$

The Growth Rate of the Six Popular Functions



Properties of Growth-Rate Functions

1. ***ignore low-order terms.***
   - If an algorithm is $O(n^3+4n^2+3n)$, it is also $O(n^3)$.
   - We only use the higher-order term as algorithm's growth-rate function.

2. ***ignore a multiplicative constant in the higher-order term***
   - If an algorithm is $O(5n^3)$, it is also $O(n^3)$.

3. ***combine growth-rate functions***
   - *$O(f(n)) + O(g(n)) = O(f(n)+g(n))$*
   - Ex: $O(n^3) + O(4n^2) \rightarrow O(n^3 +4n^2)$ ➔ So, it is $O(n^3)$.
   - Similar rules hold for multiplication.

Growth-Rate Functions

- If an algorithm takes 1 second to run with the problem size 8, what is the time requirement (approximately) for that algorithm with the problem size 16?
- If its order is:

O(1)	➔	T(n) = 1 second
O($\log_2 n$)	➔	T(n) = ($\log_2 16$) / $\log_2 8$ = 4/3 seconds
O(n)	➔	T(n) = (16) / 8 = 2 seconds
O(n*$\log_2 n$)	➔	T(n) = (16*$\log_2 16$) / 8*$\log_2 8$ = 8/3 seconds
O(n^2)	➔	T(n) = (16^2) / 8^2 = 4 seconds
O(n^3)	➔	T(n) = (16^3) / 8^3 = 8 seconds
O(2^n)	➔	T(n) = (2^{16}) / 2^8 = 2^8 seconds = 256 seconds

Contoh kasus: Insertion Sort

	cost	**time**
for j := 2 to length[A] do		
key := A[j]		
{memasukkan A[j] ke dalam array A[1…j-1] yang sudah diurutkan}		
i := j-1		
while i > 0 dan A[i] > key do		
A[i+1] := A[i]		
i := i-1		
A[i+1] := key		

tj adalah banyaknya test pada *while loop* yang dikerjakan

$$T(n) = c1(n) + c2(n-1) + c3(n-1) + c4\,(\Sigma tj) + c5\,(\Sigma(tj-1)) + c6\,(\Sigma(tj-1)) + c7(n-1)$$

$\Sigma\, i = n(n+1)/2$

$$T(n) = c1(n) + c2(n-1) + c3(n-1) + c4\,(n(n+1)/2 - 1) + c5\,(n(n-1)/2) + c6\,(n(n-1)/2) + c7(n-1)$$

$T(n) = A\,n^2 + B\,n + C = O(n^2)$

Menghitung Kompleksitas

- Suatu program yang tidak memiliki rekurensi di dalamnya akan berjalan dalam waktu konstan, tidak bergantung pada ukuran input.
- Praktisnya, perhatikan bagian loop.
  - Jika tidak ada loop: $f(n) = 1$
  - Jika ada satu loop dari 1 ... n: $f(n) = n$
  - Jika ada nested loop bersarang, masing-masing 1 ... n: $f(n) = n*n*...*n$
  - Jika ada beberapa loop yang sekuensial: $f(n)$ bergantung pada loop yang paling dominan.

Latihan

1. Rank these functions according to their growth, from slowest growing (at the top) to fastest growing (at the bottom).

n^3	n^2	n	$(3/2)^n$	2^n	1

- 1
- n
- n^2
- n^3
- $(3/2)^n$
- 2^n

2. Find the complexity!

```
void method1(int [] arr)
{
        int n = arr.length;
        for(int i = n - 1 ; i >= 0;  i = i - 3)
        {
                SOP (arr[i]);
        }
}
```

Remark: We're assuming that SOP (System.out.println) is O(1).

O(n)

Latihan

3. Find the complexity!

```
void method2(int [] arr)
{
      for(int i = 0; i <arr.length; i++)
      {
            for(int k = 0; k <arr.length; ++k)
            {
                  SOP (Math.log(arr[i]));
            }
      }
}
```

$O(n^2)$

```
void method3(int [] arr) {
       for(int i = 0; i <arr.length; i++)
       {
             method1(arr);
             method2(arr);
       }
}
```

$O(n^3)$

Latihan

4. Find the complexity

```
void method4(int [] arr)
{
      for(int i = 0; i <arr.length; i++)
      {
            for(int k = arr.length - 1;  k > 0; k = k / 3 )
            {
                  SOP (arr[i]);
            }
      }
}
```

O(n log n)

Diskusi

SOAL MENGHITUNG KOMPLEKSITAS

```
for (int i = 1; i <= 3*n; i++) {
    // code
}
```

```
for (int i = 1; i <= n+5; i++) {
    // code
}
```

```
for (int i = 1; i <= n; i += 2) {
    // code
}
```

SOAL MENGHITUNG KOMPLEKSITAS

```
for (int i = 1; i <= n; i++) {
    for (int j = i+1; j <= n; j++) {
        // code
    }
}
```

SOAL MENGHITUNG KOMPLEKSITAS

```
for (int i = 1; i <= n; i++) {
    // code
}
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        // code
    }
}
for (int i = 1; i <= n; i++) {
    // code
}
```

SOAL MENGHITUNG KOMPLEKSITAS

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        // code
    }
}
```